Group members: Ashwin Varkey 23115420

Chukwudi Williams 23061170

Manik Malik 23169717

Assignment 3

1). Peak performance and bandwidth

- Clock frequency of 2.1 GHz
- 52 cores
- AVX-512 instruction set (512-bit wide SIMD registers); each core is capable of retiring two full-width double-precision FMA instructions per cycle
- Memory: 8-channel DDR5 DRAM, 4800 MHz

(A) Calculate the double-precision floating-point peak performance of the chip in TFlops/s

Double-precision floating-point peak performance

= Super Scalarity * FMA Factor * SIMD Factor * Clock Speed * No of Cores

=2*2*8*2.1*52

= 3.4944 TFlops/sec

(B) Calculate the theoretical memory bandwidth of the chip in GByte/s

Theoretical memory bandwidth of the chip

= RAM Speed * No of Channel * No of bytes of Width

= 4800*8*8

=307.2 GB/s

(C)Consider the following "Schönauer Triad" loop on double-precision data:

Minimum time taken to do the arithmetic in the loop = No. of Flops / Peak Performance

= 2 * 10^9 Flops / 3.4944 T Flops per sec

= 0.57234 m sec

Minimum time it takes to transfer the data over the memory bus = Volume of Data / Memory Bandwidth = 4 * 8 * 10^9 Bytes/ 307.2 GBytes per sec = **104.16 m sec**

From the above calculation it is evident that the time required to transfer the data over the memory bus is the bottle neck.

2) Outer Product

2 a) The default for C++/C programming language is row by row order (row major). For continuous access the inner loop i must access rightmost array index. So in this case stride N*N access is happening during run which causes a performance problem. The problem lies in the way the elements of the array a are accessed and updated in a nested loop.

Transformed code: Loop interchange

1.
```
for(i=0; i<N; ++i)
for(j=0; j<N; ++j)

        a[i][j] = x[i] * y[j];
```

In the transformed code, the outer loop iterates over the rows of a (i), and the inner loop iterates over the columns of a (j). This ensures that the elements of x and y are accessed in a more cache-friendly manner, leading to improved performance and continuous access is occurring. The inner loop accesses rightmost index

2 b) **a[N][N]- Spatial Access locality**

y[N]- Temporal Access locality

In the case of a matrix a[N][N], spatial access locality refers to the fact that the elements are loaded continuously row-wise. This means that when accessing consecutive elements in the same row, they are likely to be stored in adjacent memory locations, which takes advantage of cache prefetching and improves data access efficiency. a[N][N]- Spatial access locality

On the other hand, the rightmost vector in the inner loop, y[N], exhibits temporal access locality. This is because, after one iteration of the inner loop, the data for y[N] is loaded into cache. In subsequent iterations, the same data from the initial load can be reused, thereby reducing memory access and improving performance.

However, if N is very large and the cache size (e.g., L1 cache) is limited, the cache line holding the data for y[N] may need to be evicted to make room for other data. This eviction can cause the **loss of temporal access locality** since the data would have to be reloaded from memory in each iteration, leading to more frequent and slower memory accesses.

2 c)

```
float a[][],x[],y[];
for(i=0; j<N; ++i)
  for(j=0; i<N; ++j)
    a[i][j] = x[i] * y[j];
```

for the corrected code(single precision floating point) we:

size of **a** = N*N*4 Bytes

size of y = N*4 Bytes

x will be treated as a temporary variable stored in the register as it only changes after N iterations. (only in each outer loop iteration)

so total data load = N*4 Bytes

data store = N^2 * 4 Bytes

total = N*4 + N^2 *4 Bytes

Code balance = data transfer / work (iteration) = (N*4 + N^2 *4) / N^2

So **4/N + 4** for sufficiently large enough N data transfer per iteration ~ 4 Bytes

If the hardware supports nontemporal stores, they can be used to reduce the memory data traffic per iteration. Nontemporal stores bypass the cache hierarchy and write the data directly to memory, avoiding unnecessary cache writes. This is beneficial when the data being written won't be reused in the near future.

There are two methods to increase temporal locality:

1. loop unroll and jam
2. Blocking/Tiling of inner (r) loop

In the second option we divide the y variable into small parts

Let’s take Nb which needs to be sufficiently small so that x and a can be loaded into some cache

```
float a[][],x[],y[];
for(it = 1; it<N ; j+= j+Nb){
 Ns = it;
 Ne = min( (j +Nb -1), N);
 for(i=0; j<N; ++i)
   for(j=Ns; i<Ne; ++j)
     a[i][j] = x[i] * y[j];
}
```

Now total data store = 4*N*N/Nb

Total data load = 4 * N/Nb

Total iterations = N*N

Code balance = (4*N*N/Nb + 4 * N/Nb)/N*N => **4/Nb + 4/(N*Nb)** ~ 4/Nb (for large N)

3) Balance

(a) s = s + a[i]; (single precision)
(b) s = s + a[i]*b[i]; (single precision)
(c) a[i][j] = b[i][j]+ s*c[i][j]; (double precision, inner loop over j)
(d) a[i] = s*a[i]; (double precision)
(e) x[i] = x[i] + s*v[index[i]]; (DP for x[] and v[], and int32 for index[])

Code balance (B_C) quantifies how much data is transfered over a given data path per unit of work:

$$B_c = \frac{\text{data transfer [byte]}}{\text{amount of work [flops, it., CL updates,...]}}$$

3 a) s = s + a[i];

No. of Instruction= 1 LOAD= 4 bytes

Flops- 1 ADD

Code Balance= 4 Bytes/Flop

3 b) s = s + a[i]*b[i];

2 LOADS =2* 4 bytes=8 bytes

No. of flops= 1 ADD and 1 MULT

Code Balance=8 bytes/2 Flop=4 Bytes/Flop

3 c) a[i][j] = b[i][j]+ s*c[i][j];

No of Instruction = 1 STORE and 2 LOAD(+1 Write allocate)

Therefore, no of bytes = 4*8 Bytes = 32 bytes

No of Flops = 2 (1 ADD, 1 MULT)

Code Balance = 32 bytes / 2 Flop = 16 Bytes/ Flop

3 d) a[i] = s*a[i];

No of Instruction = 1 LOAD, 1 STORE

Therefore, no of bytes = 2 * 8 Bytes = 16 bytes

No of Flops = 1 (1 MULT)

Code Balance = 16 bytes / 1 Flop = 16 Bytes / Flops

3 e) x[i] = x[i] + s*v [index[i]];

Best Case

When all elements of index [] is same, v need not be loaded

1 LOAD for x[i] and 1 STORE for x[i], 1 LOAD for index[i]

=2*8+4=20 Bytes

No of Flops=2(1 ADD, 1 MULT)

Code Balance=20/2=10 Bytes/Flop

Intermediate case

When the elements in index [ ] increments by 1

From best case, add another LOAD for v [index[i]]

=20+1*8=28 Bytes

Code Balance=28/2=14 Bytes/Flop

Worst Case

When the elements in index [ ] are random and cache line is not reused

No of instructions= 1 LOAD and STORE for x[i], 1 LOAD for index[i], 1 LOAD for v [index[i]]

=28+Cache Line (L)

=28+L

Code Balance=14+0.5L Bytes/Flop